PyMechanical cheat sheet

Version: main

Embedded App

Start an instance with global API entry points

```
from ansys.mechanical.core import App

app = App(version=261, globals=globals())
print(app)
```

Access entry points from Python:

```
ExtAPI    # Application.ExtAPI
DataModel # Application.DataModel
Model     # Application.DataModel.Project.Model
Tree      # Application.DataModel.Tree
Graphics  # Application.ExtAPI.Graphics
```

Open, save, and close a project

```
app.open(r"D:\Workdir\bracket.mechdb")
# Remove lockfile if the project is locked
app.open(r"D:\Workdir\bracket.mechdb",
         remove_lock=True)
app.save()
app.save_as(r"D:\Workdir\bracket_v2.mechdb",
            overwrite=True)
app.close()   # closes active project
app.new()     # resets to a new blank project
```

Print project structure as tree

```
app.print_tree()
app.print_tree(max_lines=20)  # limit output
```

Visualize in 3D (24R2+)

```
app.plot()           # plot geometry
app.plot(Model.Mesh) # plot mesh
```

Enable feature flags

```
# Available flags: 'CPython', 'ThermalShells',
# 'MultistageHarmonic'
app = App(feature_flags=["CPython",
                         "ThermalShells"])
```

Turn on logging

```
import logging
from ansys.mechanical.core import App
from ansys.mechanical.core.embedding.logger import (
    Configuration, Logger,
)

Configuration.configure(
    level=logging.WARNING, to_stdout=True
)
app = App(version=261, globals=globals())
Logger.error("Test Error Message")
```

Log from app methods:

```
app.log_info("Info message")
app.log_warning("Warning message")
app.log_error("Error message")
app.log_debug("Debug message")
```

App Helpers

Import materials

```
app.helpers.import_materials(
    r"C:\path\to\materials.xml"
)
```

Import geometry

```
app.helpers.import_geometry(
    r"C:\path\to\geometry.pmdb"
)
# With options
app.helpers.import_geometry(
    r"C:\path\to\geometry.agdb",
    process_named_selections=True,
    named_selection_key="NS",
    process_material_properties=True,
    process_coordinate_systems=True,
)
```

Export an image

```
# Export image of the geometry at 1920x1080
app.helpers.export_image(
    Model.Geometry,
    r"C:\path\to\geometry.png",
    width=1920,
    height=1080,
)
```

Execute IronPython script (embedded)

```
result = app.execute_script("2 + 3")
app.execute_script_from_file(
    r"C:\scripts\setup.py"
)
```

Access Mechanical messages

```
# Print all messages
print(app.messages)
# Access a specific message
msg = app.messages[0]
print(msg.Severity, msg.DisplayString)
```

Get project directory and version

```
print(app.project_directory)
print(app.version)
```

License Manager (v252+)

```
lm = app.license_manager
# List all licenses and their status
lm.show()
# Enable or disable a license
lm.set_license_status("mech_1", True)
lm.set_license_status("mech_1", False)
# Check a specific license
status = lm.get_license_status("ansys")
print(status)
```

Remote Session using gRPC

Launch and connect to a session

```
from ansys.mechanical.core import launch_mechanical

mechanical = launch_mechanical()
```

Launch Mechanical UI

```
mechanical = launch_mechanical(batch=False)
```

Launch using the CLI

```
ansys-mechanical -r 261 --port 10000 -g
```

Launch a specific version

```
from ansys.mechanical.core import find_mechanical

wb_exe = find_mechanical(261)[0]
mechanical = launch_mechanical(
    exec_file=wb_exe,
    verbose_mechanical=True,
    batch=True,
)
print(mechanical)
```

Connect to an existing session

```
from ansys.mechanical.core import (
    connect_to_mechanical,
)

mechanical = connect_to_mechanical(port=10000)
# Or connect remotely
mechanical = connect_to_mechanical(
    "192.168.0.1", port=10000
)
```

Run scripts on the remote session

```
result = mechanical.run_python_script("2+3")
mechanical.run_python_script(
    "Model.AddStaticStructuralAnalysis()"
)
# Run a block of commands
commands = """
materials = Model.Materials
materials.Import(r'D:\\Workdir\\copper.xml')
"""
mechanical.run_python_script(commands)
# Run from file
mechanical.run_python_script_from_file(file_path)
```

Import a project and query bodies

```
file = r"D:\\Workdir\\bracket.mechdb"
mechanical.run_python_script(
    f'DataModel.Project.Open("{file}")'
)
count = mechanical.run_python_script(
    "Model.GetChildren("
    "DataModelObjectCategory.Body, True).Count"
)
print(count)
```

Project-specific operations

```
# Project directory and file listing
print(mechanical.project_directory)
mechanical.list_files()
# Save project
mechanical.run_python_script(
    r'ExtAPI.DataModel.Project.Save("D:\\Workdir")'
)
# Logging
mechanical._log.info("Useful message.")
mechanical.log_message("INFO", "info message")
# Exit
mechanical.exit(force=True)
```